Let’s build the neural network that will serve as our Q function. As you know, in this book we’re using PyTorch for all our deep learning models, but if you’re more comfortable with another framework such as TensorFlow or MXNet, it should be fairly straightforward to port the models.

Figure 3.8 shows the general architecture for the model we will build. Figure 3.9 shows it in string diagram form with typed strings.

Figure 3.8. The neural network model we will use to play Gridworld. The model has an input layer that can accept a 64-length game state vector, some hidden layers (we use one, but two are depicted for generality), and an output layer that produces a 4-length vector of Q values for each action, given the state.

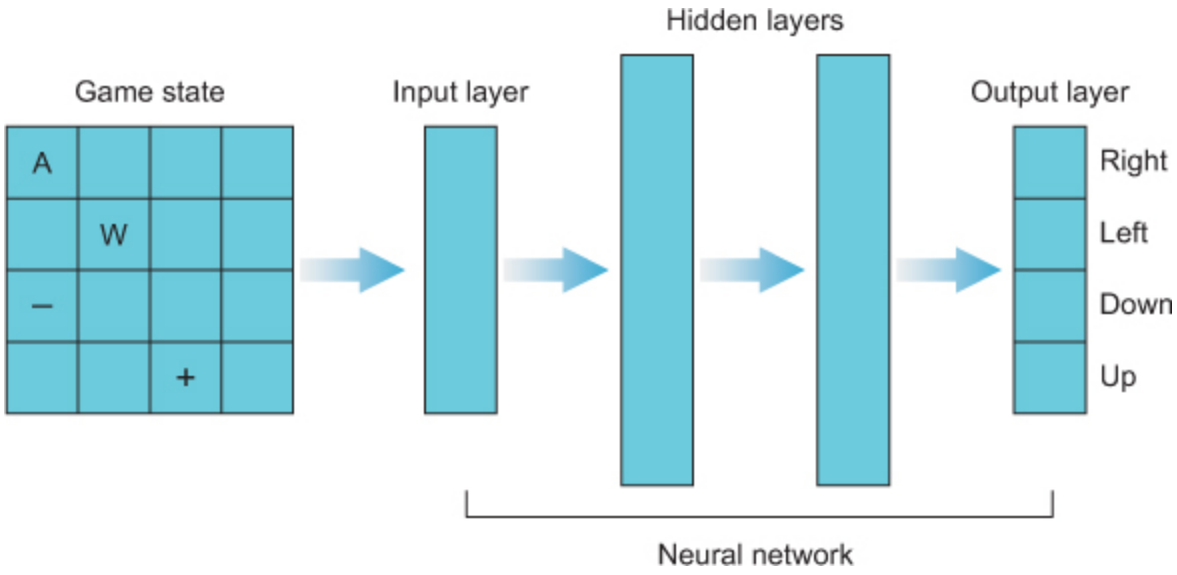


Figure 3.9. String diagram for our DQN. The input is a 64-length Boolean vector, and the output is a 4-length real vector of Q values.

